

GPU Performance — Compute-It-Yourself

Use the Formula Sheet equations to compute real numbers about your GPU — and each answer hands you an insight you can't get from a spec sheet.

By the end you'll prove, with numbers, that:

- Decode speed is set by **bandwidth**, not TFLOPs.
- The *same* GPU is memory-bound for decode and compute-bound for prefill.
- You must **batch ~100+ requests** before you actually use the compute you paid for.
- Quantization is a **throughput** lever, not just a memory one.
- On a no-NVLink box, **tensor-parallel communication can dwarf the compute**.

SYMBOL	MEANING	EXAMPLE (RTX 5090)	YOURS
C	VRAM capacity	32 GB	_____
BW	memory bandwidth	1790 GB/s	_____
P_peak	measured peak FP16 compute (Lab C)	~210 TFLOP/s	_____
W_pow	board power	~575 W	_____

A · Warm-ups — where the bottleneck lives

Q1–Q3

Q1. Decode is bandwidth, not FLOPs.

A **13B** model in FP16. At batch 1, decode must re-read *all* weights from memory for **every** token.

Find: (a) weight bytes/token; (b) max decode tok/s $\approx$ BW $\div$ weight GB.

INSIGHT

Q2. Arithmetic intensity of decode.

Decode's core op is matrix $\times$ vector: FLOPs = $2n^2$, bytes read $\approx 2n^2$ (FP16 weights).

Find: AI = FLOPs $\div$ bytes. Does it depend on model size n ?

INSIGHT

Q3. Your GPU's ridge point.

Find: Ridge = P_peak $\div$ BW (FLOP/byte). Then classify: decode (AI $\approx$ 1) and prefill matmul at $n=4096$ (AI $\approx n/2$).

INSIGHT

B · Core — throughput, batching & fit

Q4–Q8

Q4. The batching sweet spot.

A batch of **B** decode requests reuses each weight read B times, so its intensity is $AI \approx B \text{ FLOP/byte}$.

Find: how many concurrent requests **B** are needed to reach your ridge point (become compute-bound)?

INSIGHT

Q5. The quantization double-win.

Same 13B model, FP16 vs INT4.

Find: (a) weight GB each; (b) decode tok/s each ($BW \div \text{weight GB}$); (c) the speedup.

INSIGHT

Q6. KV cache vs weights.

13B model, 40 layers, hidden 5120, FP16 KV (assume MHA): $KV/\text{token} = 2 \times \text{layers} \times \text{hidden} \times 2$ bytes.

Find: KV per token, then total KV at **S = 32,000** tokens (batch 1). Compare to the 26 GB of weights.

INSIGHT

Q7. Prefill vs decode: which dominates the wait?

Prompt = 2000 tokens, generate = 500 tokens. Prefill ≈ 8000 tok/s (compute-bound); decode = your Q1 answer.

Find: prefill time and decode time. Which is bigger?

INSIGHT

Q8. Will a 34B fit? (with KV)

32 GB card, want **34B**, context 8k, batch 4, $KV \approx 0.2$ MB/token (GQA).

Find: weights at FP16 and INT4; KV for this workload; total in INT4 (+2 GB overhead). Does it fit?

INSIGHT

C · Multi-GPU & efficiency

Q9–Q11

Q9. The no-NVLink tax.

Tensor-parallel across 2 GPUs over PCIe (measured link = **50 GB/s**). Per layer you all-reduce activations bytes = $B \times S \times H \times D = 1 \times 2048 \times 4096 \times 2$. Model has 32 layers.

Find: communication time per layer, then per token ($\times 32$). Decode does almost no compute per token.

INSIGHT

Q10. Two 5090s: split it or double it?

A 13B model fits on **one** 32 GB card (26 GB). You have 2×5090 , **no NVLink**.

Find: compare (A) tensor-split (pays the Q9 tax every token) vs (B) data-parallel (full model per card, 2 independent streams). Aggregate tok/s for B?

INSIGHT

Q11. Tokens per watt (stretch).

Throughput 69 tok/s at 575 W.

Find: tokens/joule; energy for 1M tokens (kWh); cost at \$0.05/kWh. What happens to tokens/watt when you batch?

INSIGHT

✓ Answer Key — worked

Uses the RTX 5090 example column. Redo each with your own numbers — the insights hold, only the magnitudes change.

Q1

weights = $13 \times 2 = 26$ GB/token · decode $\approx 1790 \div 26 = \sim 69$ tok/s.

Even a card doing hundreds of TFLOPs generates only ~69 tok/s. Decode speed = bandwidth ÷ model size — compute barely matters.

Q2

AI = $2n^2 \div 2n^2 = 1$ FLOP/byte, independent of n.

Decode is ~1 FLOP/byte for any model size → always deep in memory-bound territory.

Q3

Ridge = $210 \times 10^{12} \div 1790 \times 10^9 = \sim 117$ FLOP/byte. Decode $1 \ll 117 \rightarrow$ **memory-bound**; prefill $\sim 2048 \gg 117 \rightarrow$ **compute-bound**.

The same GPU is memory-bound for decode and compute-bound for prefill — one chip, two regimes.

Q4

Need AI $\geq$ Ridge $\rightarrow B \geq \sim 117$ requests.

You must batch ~117 concurrent requests before the compute you paid for is actually used. Below that, the Tensor Cores idle — this is why serving = batching.

Q5

FP16 = 26 GB $\rightarrow 1790/26 = 69$ tok/s. INT4 = 6.5 GB $\rightarrow 1790/6.5 = \sim 275$ tok/s. Speedup = $\sim 4\times$.

INT4 doesn't just fit ~4x smaller — it decodes ~4x faster, because decode reads ~4x fewer bytes/token. Quantization is a throughput lever.

Q6

KV/token = $2 \times 40 \times 5120 \times 2 = 819,200$ B ≈ 0.8 MB. At 32k tokens $\rightarrow 0.8$ MB $\times 32768 \approx \sim 26$ GB — equal to the weights.

At long context the KV cache can match or exceed the model itself. Capacity planning that ignores KV will OOM. (GQA cuts this several-fold — check kv_heads.)

Q7

Prefill = $2000/8000 = 0.25$ s. Decode = $500/69 = \sim 7.25$ s. Decode is $\sim 29\times$ longer.

For chat, the wait is generation, not prompt-reading. Optimize decode (bandwidth, batching, quantization) — TTFT is already small.

Q8

FP16 = 68 GB (**no fit**). INT4 = 17 GB. KV = $0.2 \text{ MB} \times 8192 \times 4 = \mathbf{6.6 \text{ GB}}$. Total INT4 = $17 + 6.6 + 2 = \mathbf{\sim 25.6 \text{ GB} \leq 32 \rightarrow \text{fits}}$.

A 34B fits on 32 GB only in INT4, and only after budgeting KV for your batch $\times$ context — the fit depends on the workload, not just the model.

Q9

bytes = 16.78 MB. Comm/layer = $16.78 \times 10^6 \div 50 \times 10^9 = \mathbf{0.336 \text{ ms}}$. Per token $\times 32 = \mathbf{\sim 10.7 \text{ ms}}$ of pure communication.

$\sim 10.7 \text{ ms/token}$ just moving data — far more than the tiny per-token compute. On PCIe, tensor-parallel is communication-bound.

Q10

(A) tensor-split: pays $\sim 10.7 \text{ ms/token} \rightarrow$ slower than one GPU. (B) data-parallel: two full models, ~ 0 cross-GPU traffic $\rightarrow \mathbf{2 \times 69 = \sim 138 \text{ tok/s}}$ aggregate.

If a model fits on one card, two cards should double throughput (data-parallel), not be glued into one bigger-but-slower model.

Q11

$69 \div 575 = \mathbf{0.12 \text{ tok/J}}$. 1M tokens $\rightarrow 1\text{e}6/0.12 \approx 8.3 \text{ MJ} = \mathbf{\sim 2.3 \text{ kWh}} \rightarrow \mathbf{\sim \$0.12}$ (energy only). Batching raises tokens/watt.

Energy per token is tiny; the real cost is the GPU itself. But tokens/watt still ranks efficiency — and batching is the cheapest way to raise it.

Companion to the Tasar'u GPU Infrastructure Labs · swap in your own measured numbers and the insights still hold — only the magnitudes change.